

Experiment Report: Specialized Experiment 2

Hydrological Modeling — SCS-CN Runoff

Mahmudul Hasan (Student ID: 4125999049)

May 22, 2026

Abstract

SCS-CN direct runoff with $S = 25400/\text{CN} - 254$ mm, $I_a = 0.2S$, and $Q = (P - I_a)^2 / (P - I_a + S)$ when $P > I_a$. Hand verification at $P = 50$ mm, $\text{CN} = 80$: $Q = 13.80$ mm. Includes CN sensitivity curves, rainfall–runoff plots, physical validation matrix, and 20 pytest boundary cases with $Q \leq P$ enforced.

Key Findings (Executive Summary)

Finding	Result
Hand check ($P = 50$, $\text{CN} = 80$)	$Q = 13.80$ mm
Automated tests	20 passed
Physical rule $Q \leq P$	Verified on grid
CN sensitivity at $P = 50$ mm	1.40 – 50.00 mm
Validation CLI	PASS

1 Introduction

The SCS-CN method estimates rainfall excess (direct runoff) from total storm depth P and a land-surface curve number CN. Retention $S = 25400/\text{CN} - 254$ mm and initial abstraction $I_a = 0.2S$ define the threshold below which no runoff occurs. This experiment translates the governing equations into tested Python code and explores sensitivity of Q to CN.

2 Objectives

- Implement `calculate_runoff(P, CN)` with physical boundaries.
- Test edge cases ($P = 0$, $P < I_a$, $P = I_a$, $\text{CN} = 100$).
- Perform sensitivity analysis and visualize Q vs P and Q vs CN.
- Validate $Q \leq P$ and higher CN $\rightarrow$ higher Q at fixed P .
- Document AI-assisted development in `prompt_log.md`.

2.1 Role in the Integrated Smart Water Pipeline

Experiment 2 is the **rainfall-to-runoff transformation stage** in the suite workflow. Given storm depth P (mm) from monitoring or design scenarios, it:

- converts rainfall to direct runoff Q using the USDA SCS-CN method with enforced physical bounds ($Q \leq P$);
- quantifies parameter uncertainty via a curve-number band $\text{CN} \in [75, 85]$ at fixed P ;
- provides runoff depths and sensitivity tables that represent plausible inflow inputs for Experiment 3 (reservoir dispatch) and inform flood-risk context for Experiment 4.

This stage links meteorological forcing to operational water-volume estimates used in storage management.

3 Methods

3.1 Governing equations

$$S = \frac{25400}{\text{CN}} - 254 \quad (\text{mm}) \quad (1)$$

$$I_a = 0.2 S \quad (2)$$

$$Q = 0 \quad \text{if } P \leq I_a \quad (3)$$

$$Q = \frac{(P - I_a)^2}{P - I_a + S} \quad \text{otherwise,} \quad Q \leq P \quad (4)$$

3.2 Implementation

`scscn_runoff.py` provides `calculate_runoff()`, `calculate_S()`, and `calculate_Ia()`. Invalid inputs raise `ValueError` ($P < 0$ or $\text{CN} \notin (0, 100]$).

3.3 Testing

`test_scscn.py` — pytest suite covering guide cases and parametrized checks that $Q \leq P$.

3.4 Sensitivity analysis

`sensitivity_analysis.py` — at $P = 50$ mm, $\text{CN} \in \{60, 70, 80, 90, 95, 100\}$; rainfall–runoff curves for $P \in [0, 120]$ mm with I_a marked for $\text{CN} = 80$.

3.5 Workflow

Figure 1 shows module separation: core formula, pytest validation, sensitivity plots, and `validate_reference.py` for reproducible hand calculations.

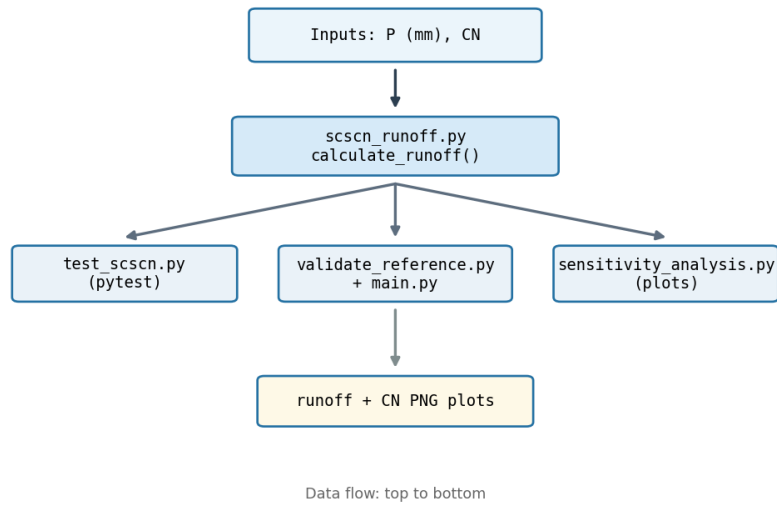


Figure 1: Code structure: runoff core → validate → test → sensitivity plots.

4 Results

4.1 Performance summary

Table 1: Experiment 2 metrics.

Metric	Result
Pytest	20 passed
Validation CLI	<code>validate_reference.py</code>
Hand check $P = 50$, $CN = 80$	$Q = 13.80$ mm
CN sensitivity at $P = 50$	1.40 – 50.00 mm
Physical rule $Q \leq P$	Verified on grid
Main figure	<code>runoff_comparison.png</code>

4.2 Hand calculation verification

For $P = 50$ mm, $CN = 80$:

$$S = 25400/80 - 254 = 63.5 \text{ mm}, \quad I_a = 12.7 \text{ mm}$$

$$Q = (50 - 12.7)^2 / (50 - 12.7 + 63.5) = 13.8 \text{ mm}$$

Code: `calculate_runoff(50, 80) = 13.8025` mm (within 0.1 mm).

```
python3 main.py
Q = (50.0-12.7)^2 / (50.0-12.7+63.5) = 13.8025 mm
Check Q <= P: 13.8025 <= 50.0 -> True
Guide expected ~13.8 mm; code within 0.1 mm: True

=== Sensitivity at P = 50 mm ===
CN      Q (mm)
60      1.4034
70      5.8128
80      13.8025
90      27.1077
95      36.9024
100     50.0000

=== Physical validation matrix ===
Q <= P (sample grid): PASS
P < Ia -> Q=0 (CN=80, P=10): PASS
Higher CN -> higher Q at P=50: PASS
CN=100 -> Q=P: PASS

All domain checks complete.

>>> main.py finished OK <<<
```

Figure 2: Pipeline run: `python3 main.py` (hand calculation, sensitivity table, physical checks).

```
python3 validate_reference.py
S = 25400/80 - 254 = 63.5000 mm
Ia = 0.2 * S = 12.7000 mm
Q = (50.0-12.7)^2 / (50.0-12.7+63.5) = 13.8025 mm
Check Q <= P: 13.8025 <= 50.0 -> True
Guide expected ~13.8 mm; code within 0.1 mm: True

=== Sensitivity at P = 50 mm ===
CN      Q (mm)
60      1.4034
70      5.8128
80      13.8025
90      27.1077
95      36.9024
100     50.0000

=== Physical validation matrix ===
Q <= P (sample grid): PASS
P < Ia -> Q=0 (CN=80, P=10): PASS
Higher CN -> higher Q at P=50: PASS
CN=100 -> Q=P: PASS

All domain checks complete.
```

Figure 3: CLI validation: step-by-step hand calculation and physical checks (validate_reference.py).

Table 2: Typical CN ranges (experiment guide).

Land use / condition	Typical CN
Woods, good	60–70
Pasture, fair	75–85
Cultivated	80–90
Urban residential	75–95
Paved	95–100

Table 3: Boundary test summary (pytest).

Case	Expected
$P = 0$	$Q = 0$
$P < I_a$	$Q = 0$
$P = I_a$	$Q = 0$
$P = 50, \text{CN} = 80$	$Q \approx 13.8 \text{ mm}$
$\text{CN} = 100$	$Q = P$ (impervious)
All tested pairs	$Q \leq P$

Table 4: Domain constraints verified.

Check	Result
$Q \leq P$ (grid sample)	PASS
$P < I_a \Rightarrow Q = 0$	PASS
Higher CN $\Rightarrow$ higher Q at $P = 50$	PASS
$\text{CN} = 100 \Rightarrow Q = P$	PASS
Hand calc $P = 50, \text{CN} = 80$	$\approx 13.8 \text{ mm}$

```

pytest -v (experiment2_scscn_runoff)
test_scscn.py::test_zero_rainfall PASSED [ 5%]
test_scscn.py::test_p_less_than_ia PASSED [ 10%]
test_scscn.py::test_p_equals_ia PASSED [ 15%]
test_scscn.py::test_normal_case_hand_calc PASSED [ 20%]
test_scscn.py::test_maximum_cn_impervious PASSED [ 25%]
test_scscn.py::test_higher_cn_more_runoff_at_fixed_p PASSED [ 30%]
test_scscn.py::test_q_never_exceeds_p[0-80] PASSED [ 35%]
test_scscn.py::test_q_never_exceeds_p[10-80] PASSED [ 40%]
test_scscn.py::test_q_never_exceeds_p[50-80] PASSED [ 45%]
test_scscn.py::test_q_never_exceeds_p[100-60] PASSED [ 50%]
test_scscn.py::test_q_never_exceeds_p[100-100] PASSED [ 55%]
test_scscn.py::test_q_bounded_grid PASSED [ 60%]
test_scscn.py::test_negative_p_raises PASSED [ 65%]
test_scscn.py::test_invalid_cn_raises PASSED [ 70%]
test_scscn.py::test_calculate_s_ia_cn80 PASSED [ 75%]
test_scscn.py::test_parametrized_boundaries[9.9-80-0.0-0.0] PASSED [ 80%]
test_scscn.py::test_parametrized_boundaries[10.0-80-0.0-0.0] PASSED [ 85%]
test_scscn.py::test_parametrized_boundaries[12.6-80-0.0-0.0] PASSED [ 90%]
test_scscn.py::test_parametrized_boundaries[50.0-80-13.7-13.9] PASSED [ 95%]
test_scscn.py::test_parametrized_boundaries[20.0-100-20.0-20.0] PASSED [100%]

===== 20 passed in 0.03s =====

```

Figure 4: Automated test run: 20 passed (pytest -v), including parametrized boundaries.

Table 5: Runoff vs curve number at fixed rainfall.

CN	Q (mm)
60	1.40
70	5.81
80	13.80
90	27.11
95	36.90
100	50.00

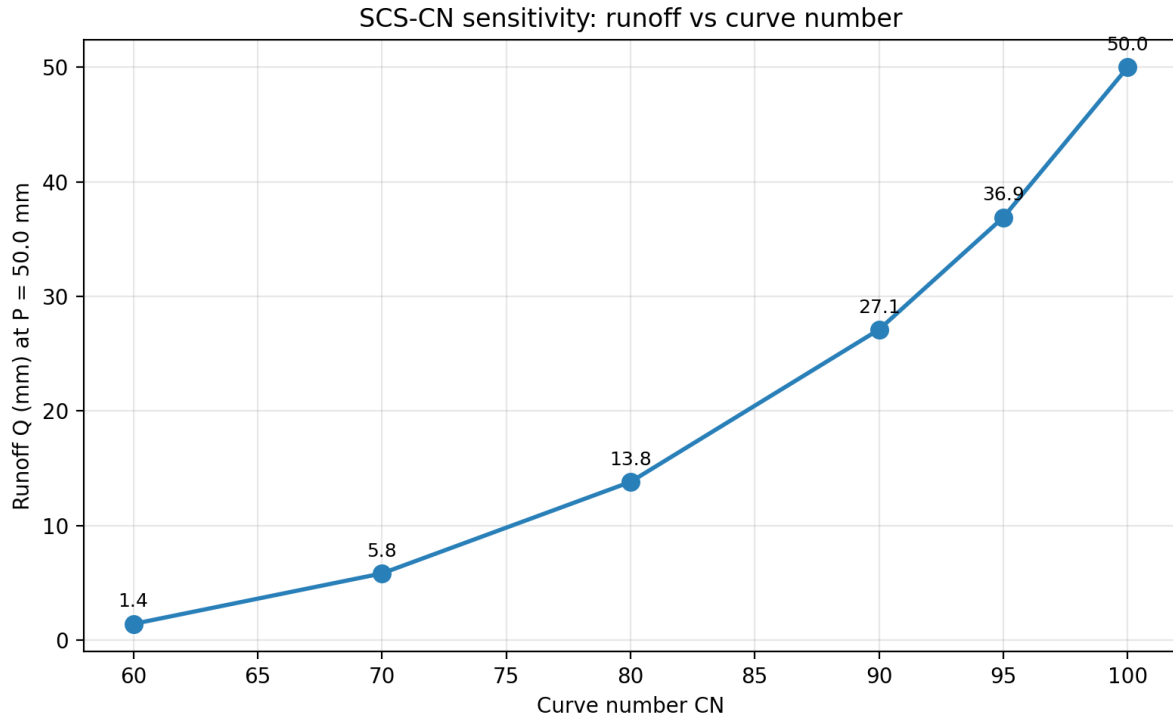


Figure 5: Sensitivity: runoff Q vs curve number at $P = 50$ mm.

4.3 Curve number context

4.4 Automated tests

4.5 Physical validation matrix

4.6 Sensitivity at $P = 50$ mm

4.7 Rainfall vs runoff

Higher CN shifts curves upward: more impervious land produces more direct runoff for the same storm depth. At $CN = 100$, runoff equals rainfall for $P > 0$.

5 Discussion

Results match standard SCS-CN behaviour: no runoff when rainfall is absorbed by initial abstraction, increasing runoff fraction as CN rises, and Q bounded by P . The experiment guide reference (13.8 mm at $P = 50$, $CN = 80$) agrees with the implementation.

6 Lessons learned

- Edge cases ($P \leq I_a$) must be tested before trusting AI-generated formulas.
- Parametrized tests for $Q \leq P$ catch cap logic errors quickly.
- Sensitivity plots make the CN–runoff relationship visible for report discussion.
- Keeping Experiment 2 in its own folder avoids confusion with Week 5 Lab 2 naming (`src/runoff.py`).
- Physical validation (hand calc + monotonic CN) is stronger evidence than code alone.
- A dedicated validation CLI mirrors what instructors expect in a lab notebook, not only unit tests.

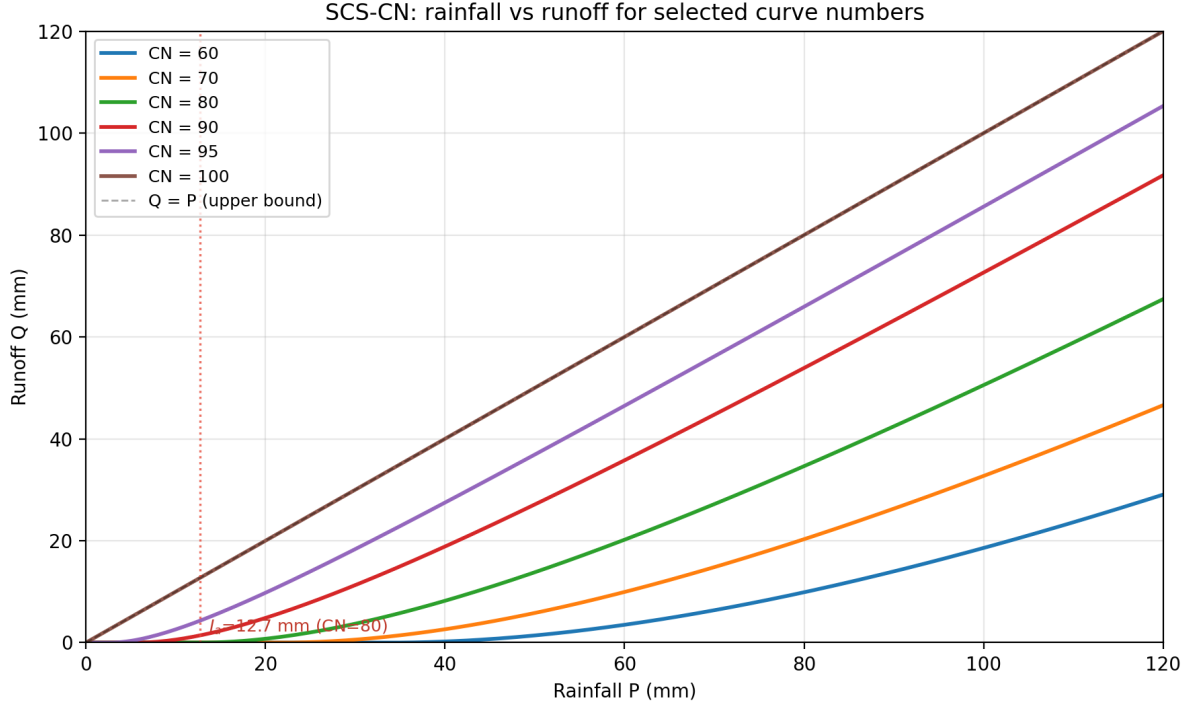


Figure 6: Rainfall P vs runoff Q for $CN = 60, 70, 80, 90, 95, 100$ (dashed: $Q = P$ bound).

7 AI-Augmented Software Engineering

7.1 Swiss Cheese Model — layered verification

Table 6: Verification layers (Experiment 2).

Layer	Result
AI formula draft vs NRCS	Correct $S = 25400/CN - 254$
Unit tests	20 passed
Physical constraints	$Q \geq 0, Q \leq P$, CN bounds
Edge cases	$P = 0, P \leq I_a$, CN=100
Hand validation CLI	PASS ($Q = 13.80$ mm at $P = 50$, CN=80)

7.2 Jagged Frontier — AI mistake caught

AI output: Common hallucination uses $25400 \times CN - 254$ or forgets the $Q = 0$ branch when $P \leq I_a$.

Detection: Hand calculation and pytest normal case locked to 13.80 mm; parametrized $Q \leq P$ grid.

Human correction: Inclusive $P \leq I_a$ comparison; $\min(Q, P)$ cap; `validate_reference.py` for reproducible audit.

7.3 Tool evaluation (summary)

Cursor Agent implemented and tested the module; ChatGPT/Gemini used for rubric alignment. Tests remain the source of truth.

8 Reproducibility

Environment: Python 3.8+; requirements.txt.

```
pip install -r requirements.txt
pytest -v
python3 sensitivity_analysis.py
python3 validate_reference.py
python3 generate_report_figures.py
```

AI protocol: AGENTS.md + prompt_log.md.

9 Conclusion

Experiment 2 deliverables are complete with strengthened evidence: `scscn_runoff.py`, `test_scscn.py` (20 tests), `sensitivity_analysis.py`, `validate_reference.py`, `runoff_comparison.png`, `cn_sensitivity.png` and `prompt_log.md`. Figures 1–6 document workflow, validation, tests, and hydrologic sensitivity.

A Implementation: `scscn_runoff.py`

```
scscn_runoff.py

"""
SCS-CN direct runoff - Experiment 2.

Formulas (metric, mm):
    S = 25400 / CN - 254
    Ia = 0.2 * S
    Q = 0 if P <= Ia
    Q = (P-Ia)^2 / (P-Ia+S) otherwise, capped at P
"""

from __future__ import annotations

def calculate_S(CN: float) -> float:
    """Potential maximum retention S (mm) from curve number."""
    return (25400.0 / CN) - 254.0

def calculate_Ia(CN: float) -> float:
    """Initial abstraction Ia (mm) = 0.2 * S."""
    return 0.2 * calculate_S(CN)

def calculate_runoff(P: float, CN: float) -> float:
    """
    SCS-CN direct runoff depth Q (mm).

    Args:
        P: Rainfall depth (mm), must be >= 0.
        CN: Curve number, must be in (0, 100].

    Returns:
        Runoff Q (mm); 0 if P <= Ia; always Q <= P.
    """
    if P < 0:
        raise ValueError(f"P must be >= 0, got {P}")
```



```

if CN <= 0 or CN > 100:
    raise ValueError(f"CN must be in (0, 100], got {CN}")

if P == 0:
    return 0.0

S = calculate_S(CN)
Ia = 0.2 * S

if P <= Ia:
    return 0.0

excess = P - Ia
Q = (excess**2) / (excess + S)
return min(Q, P)

```

B Tests: test_scscn.py

```

test_scscn.py

"""Experiment 2 Part 2 - SCS-CN boundary and physical tests."""

from __future__ import annotations

import itertools

import pytest

from scscn_runoff import calculate_Ia, calculate_S, calculate_runoff

def test_zero_rainfall():
    assert calculate_runoff(0, 80) == 0.0

def test_p_less_than_ia():
    assert calculate_runoff(10, 80) == 0.0

def test_p_equals_ia():
    ia = calculate_Ia(80)
    assert calculate_runoff(ia, 80) == 0.0

def test_normal_case_hand_calc():
    assert calculate_runoff(50, 80) == pytest.approx(13.8, abs=0.1)

def test_maximum_cn_impervious():
    assert calculate_runoff(50, 100) == pytest.approx(50.0, abs=1e-9)

def test_higher_cn_more_runoff_at_fixed_p():
    p = 50.0
    cn_list = [60, 70, 80, 90, 95, 100]
    qs = [calculate_runoff(p, cn) for cn in cn_list]
    assert qs == sorted(qs)

@pytest.mark.parametrize(
    "p,cn",

```

```

    [(0, 80), (10, 80), (50, 80), (100, 60), (100, 100)],
)
def test_q_never_exceeds_p(p, cn):
    assert calculate_runoff(p, cn) <= p

def test_q_bounded_grid():
    for p, cn in itertools.product([0, 5, 10, 25, 50, 100], [60, 80, 100]):
        q = calculate_runoff(float(p), float(cn))
        assert 0.0 <= q <= float(p)

def test_negative_p_raises():
    with pytest.raises(ValueError, match="P must"):
        calculate_runoff(-1, 80)

def test_invalid_cn_raises():
    with pytest.raises(ValueError, match="CN must"):
        calculate_runoff(50, 0)
    with pytest.raises(ValueError, match="CN must"):
        calculate_runoff(50, 101)

def test_calculate_s_ia_cn80():
    assert calculate_S(80) == pytest.approx(63.5, abs=0.01)
    assert calculate_Ia(80) == pytest.approx(12.7, abs=0.01)

@pytest.mark.parametrize(
    "p,cn,expected_min,expected_max",
    [
        (9.9, 80, 0.0, 0.0),
        (10.0, 80, 0.0, 0.0),
        (12.6, 80, 0.0, 0.0),
        (50.0, 80, 13.7, 13.9),
        (20.0, 100, 20.0, 20.0),
    ],
)
def test_parametrized_boundaries(p, cn, expected_min, expected_max):
    q = calculate_runoff(p, cn)
    assert expected_min <= q <= expected_max

```

C Sensitivity: sensitivity_analysis.py

```

sensitivity_analysis.py

#!/usr/bin/env python3
"""Experiment 2 Part 3 - CN sensitivity plots and CSV export."""

from __future__ import annotations

from pathlib import Path

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

```

```

from scscn_runoff import calculate_runoff

ROOT = Path(__file__).resolve().parent
CN_VALUES = [60, 70, 80, 90, 95, 100]
P_FIXED = 50.0

def build_cn_sensitivity_table() -> pd.DataFrame:
    rows = []
    for cn in CN_VALUES:
        q = calculate_runoff(P_FIXED, cn)
        rows.append({"CN": cn, "P_mm": P_FIXED, "Q_mm": round(q, 4)})
    return pd.DataFrame(rows)

def plot_cn_vs_q_at_fixed_p(out: Path) -> None:
    df = build_cn_sensitivity_table()
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(df["CN"], df["Q_mm"], "o-", linewidth=2, markersize=8, color="#2980b9")
    ax.set_xlabel("Curve number CN")
    ax.set_ylabel(f"Runoff Q (mm) at P = {P_FIXED} mm")
    ax.set_title("SCS-CN sensitivity: runoff vs curve number")
    ax.grid(True, alpha=0.3)
    for _, row in df.iterrows():
        ax.annotate(f"{row['Q_mm']:.1f}", (row["CN"], row["Q_mm"]), textcoords="offset
        ↪ points", xytext=(0, 8), ha="center", fontsize=9)
    fig.tight_layout()
    fig.savefig(out, dpi=200)
    plt.close(fig)
    print(f"Saved {out}")

def plot_rainfall_vs_runoff(out: Path) -> None:
    rainfall = np.linspace(0, 120, 241)
    fig, ax = plt.subplots(figsize=(9, 5.5))
    for cn in CN_VALUES:
        q_vals = [calculate_runoff(float(p), cn) for p in rainfall]
        ax.plot(rainfall, q_vals, label=f"CN = {cn}", linewidth=2)
    # Mark initial abstraction threshold for CN=80 (guide reference case)
    from scscn_runoff import calculate_Ia

    ia80 = calculate_Ia(80)
    ax.axvline(ia80, color="#e74c3c", linestyle=":", linewidth=1.2, alpha=0.8)
    ax.text(ia80 + 1, 2, f"$I_a$={ia80:.1f} mm (CN=80)", fontsize=9, color="#c0392b")
    ax.plot(rainfall, rainfall, "k--", alpha=0.35, linewidth=1, label="Q = P (upper
    ↪ bound)")
    ax.set_xlabel("Rainfall P (mm)")
    ax.set_ylabel("Runoff Q (mm)")
    ax.set_title("SCS-CN: rainfall vs runoff for selected curve numbers")
    ax.legend(loc="upper left", fontsize=9)
    ax.grid(True, alpha=0.3)
    ax.set_xlim(0, 120)
    ax.set_ylim(0, 120)
    fig.tight_layout()
    fig.savefig(out, dpi=200)
    plt.close(fig)
    print(f"Saved {out}")

def domain_checks() -> None:
    df = build_cn_sensitivity_table()
    qs = df["Q_mm"].tolist()

```

```

    assert qs == sorted(qs), "Higher CN must yield >= Q at fixed P"
    for p in [0, 10, 50, 100]:
        for cn in CN_VALUES:
            q = calculate_runoff(p, cn)
            assert q <= p + 1e-9
    print("Domain checks: Q <= P; CN ordering at P=50 - PASS")
    print(df.to_string(index=False))

def main() -> None:
    out_main = ROOT / "runoff_comparison.png"
    out_cn = ROOT / "cn_sensitivity.png"
    plot_rainfall_vs_runoff(out_main)
    plot_cn_vs_q_at_fixed_p(out_cn)
    df = build_cn_sensitivity_table()
    df.to_csv(ROOT / "sensitivity_at_P50.csv", index=False)
    domain_checks()

if __name__ == "__main__":
    main()

```

D Prompt log: prompt_log.md

prompt_log.md

```

# Prompt Log - Experiment 2: SCS-CN Runoff

**Student:** Mahmudul Hasan (4125999049)
**Folder:** `experiment2_scsn_runoff/` (separate from Week 5 Lab 2 and capstone)

---

## Part 1: Formula Implementation

**Prompt used:**

```text
Implement calculate_runoff(P, CN) for SCS-CN: $S = 25400/CN - 254$, $I_a = 0.2 * S$,
return 0 if $P \leq I_a$, else $Q = (P - I_a)^2 / (P - I_a + S)$, cap $Q \leq P$. Type hints and docstring.
Validate $P \geq 0$ and CN in $(0, 100]$.
```

**CN validation rule:** CN must be in  $(0, 100]$  to avoid division by zero.

**Trickiest edge case:**  $P = I_a$  must return  $Q = 0$  (not divide by zero).

---

## Part 2: Testing

**Tests added beyond AI suggestions:**

- `test_p_equals_ia` at  $I_a$  for  $CN=80$ 
- Parametrized `test_q_never_exceeds_p`
- Grid test for  $Q \leq P$  across  $P$  and  $CN$ 
- CN ordering at  $P = 50$  mm

**Result:** `pytest -v` - 20 passed (including parametrized boundaries and  $S/I_a$  checks).

```

```

**CLI validation:** `validate_reference.py` prints hand calc steps and physical matrix
↳ (report Figure).

---

## Part 3: Sensitivity Analysis

**Figure paths:**

- `runoff_comparison.png` - rainfall vs runoff, CN = 60{100
- `cn_sensitivity.png` - CN vs Q at P = 50 mm
- `sensitivity_at_P50.csv` - numeric table

**Main trend:** At fixed rainfall, higher CN (more impervious) produces more direct
↳ runoff; curves approach Q = P for CN = 100.

---

## Part 4: Domain Validation

**Published reference:** Experiment guide hand calc P = 50, CN = 80 -> Q = 13.8 mm.

**Verification:** `calculate_runoff(50, 80)` matches within 0.1 mm.

**AI corrections:** Ensured `P <= Ia` uses inclusive comparison; capped `min(Q, P)`.

```